

📖 Complete Core Java Topics (Latest & Comprehensive)

1. Introduction to Java

- History & Features of Java
 - JVM, JRE, and JDK
 - Platform Independence
 - Java Program Structure
 - How Java Works (Compilation & Execution)
-

2. Data Types, Variables & Operators

- Primitive & Non-Primitive Data Types
 - Type Casting
 - Operators (Arithmetic, Logical, Bitwise, etc.)
 - Constants and Literals
-

3. Control Flow Statements

- Conditional Statements (`if`, `if-else`, `switch`)
 - Looping Statements (`for`, `while`, `do-while`)
 - Jump Statements (`break`, `continue`, `return`)
-

4. Arrays

- Single Dimensional Arrays
 - Multi-Dimensional Arrays
 - Array Class and Methods
 - For-each Loop
-

5. Object-Oriented Programming (OOP)

- Classes and Objects
- Constructors
- `this` Keyword

- Inheritance
 - Method Overloading & Overriding
 - `super` Keyword
 - `final` Keyword
 - Static Members
 - Access Modifiers (`public`, `private`, etc.)
-

6. Encapsulation, Abstraction, Polymorphism

- Abstract Classes
 - Interfaces
 - Compile-time & Runtime Polymorphism
-

7. String Handling

- `String`, `StringBuilder`, `StringBuffer`
 - String Methods
 - String Comparison
 - String Immutability
-

8. Exception Handling

- Types of Exceptions
 - Try, Catch, Finally
 - Throw vs Throws
 - Custom Exceptions
-

9. Wrapper Classes

- Autoboxing and Unboxing
 - Primitive to Object Conversion and Vice Versa
-

10. Java Input/Output (I/O)

- Byte Stream and Character Stream

- File Handling (`File`, `FileReader`, `FileWriter`)
 - Buffered Streams
 - Serialization and Deserialization
-

11. Collections Framework

- List (`ArrayList`, `LinkedList`)
 - Set (`HashSet`, `LinkedHashSet`, `TreeSet`)
 - Map (`HashMap`, `TreeMap`, `LinkedHashMap`)
 - Queue, Stack
 - Iterators: `Iterator`, `ListIterator`, `Enumeration`
 - Collections Utility Class
-

12. Generics

- Generic Classes and Methods
 - Bounded Type Parameters
 - Wildcards (`? extends`, `? super`)
-

13. Multithreading and Concurrency

- Thread Life Cycle
 - Thread Class & Runnable Interface
 - Thread Synchronization
 - Inter-thread Communication
 - `wait()`, `notify()`, `join()`, `sleep()`
 - Executor Framework (basic)
-

14. Java 8+ Features (Important for Interviews & Real-World)

- Lambda Expressions
 - Functional Interfaces (`Predicate`, `Function`, `Supplier`, etc.)
 - Stream API
 - Method References
 - Default & Static Methods in Interfaces
 - Optional Class
-

15. Inner Classes

- Non-static Inner Class
 - Static Nested Class
 - Local Inner Class
 - Anonymous Inner Class
-

16. Java Packages & Modifiers

- Creating and Using Packages
 - `import` statement
 - `protected`, `default` access
-

17. Java Annotations (Basics)

- Built-in Annotations (`@Override`, `@Deprecated`)
 - Custom Annotations (Intro only in Core)
-

18. Java Date and Time API (Java 8)

- `LocalDate`, `LocalTime`, `LocalDateTime`
 - `Period`, `Duration`
 - Formatting and Parsing Dates (`DateTimeFormatter`)
-

19. Java Utility Classes

- `Math`
 - `Arrays`
 - `Collections`
 - `Objects`
 - `Scanner`, `Random`, `StringTokenizer`
-

20. Enum in Java

- Enum Basics

- Enum with Constructor and Methods
-

21. Static and Instance Initialization

- Static Initializer Blocks
 - Instance Initializer Blocks
-

22. Java Applets (Deprecated but useful in legacy)

- Applet Basics
 - Lifecycle Methods
-

23. Command Line Arguments

- Reading arguments in `main(String[] args)`
-

⚡ Bonus: Best Practices and Coding Standards

- Code readability
 - Naming conventions
 - Use of comments and documentation
-

🔗 Step-by-Step Java Learning Path After Core Java

✔ Step 1: JDBC (Java Database Connectivity)

Learn how Java connects to databases like MySQL/PostgreSQL

- JDBC Architecture
- `DriverManager`, `Connection`, `Statement`, `PreparedStatement`, `ResultSet`
- Connecting to MySQL
- Executing SQL queries from Java
- Transaction Management (commit, rollback)
- Batch Processing

✔Step 2: Servlet Technology

Foundation for understanding web apps & request-response cycle

- Web Architecture (Client-Server Model)
- HTTP Protocol Basics (GET, POST)
- Servlet Lifecycle
- `HttpServlet`, `doGet()`, `doPost()`
- `ServletConfig` & `ServletContext`
- Session Management (Cookies, `HttpSession`)
- `RequestDispatcher`, Redirecting

✔Step 3: JSP (JavaServer Pages)

HTML + Java = Dynamic web pages

- JSP Syntax, Directives
- Scriptlets, Declarations, Expressions
- Implicit Objects (`request`, `response`, `session`)
- Expression Language (EL)
- JSTL (JSP Standard Tag Library)
- Form handling

✔Step 4: MVC Architecture with Servlets and JSP

Helps build organized, maintainable web apps

- Model, View, Controller separation
- MVC Project Structure
- Small Web App using MVC with JDBC

✔Step 5: Maven

Java Project Build & Dependency Management

- What is Maven?
- `pom.xml` and dependencies

- Build Life Cycle (clean, compile, package)
 - Creating and running Maven Projects
-

✔Step 6: Hibernate ORM

Simplify working with databases

- ORM Concepts
 - Hibernate Configuration (XML & Annotation)
 - Mapping Entity Classes to Tables
 - CRUD with Hibernate
 - HQL (Hibernate Query Language)
 - Relationships (OneToOne, OneToMany, ManyToMany)
 - Caching and Transactions
-

✔Step 7: Spring Framework

Most popular framework for building secure, fast Java apps

Spring Core

- Dependency Injection (DI)
 - Spring Beans, Bean Lifecycle
 - ApplicationContext vs BeanFactory
 - Spring Annotations (@Component, @Autowired, etc.)
-

✔Step 8: Spring MVC

Build Web Apps using Spring Framework

- DispatcherServlet Flow
 - Request Mapping (@Controller, @RequestMapping)
 - Form Handling
 - View Resolvers
 - REST APIs with Spring MVC
-

✔Step 9: Spring Boot

Fast way to build production-grade Java applications

- What is Spring Boot?
 - Spring Initializr
 - Auto-configuration
 - Embedded Tomcat
 - REST API with Spring Boot
 - Project Structure
 - Exception Handling (@ControllerAdvice)
 - CRUD using Spring Boot + JPA + MySQL
-

✔Step 10: RESTful Web Services

API development for web/mobile apps

- REST vs SOAP
 - @RestController, @GetMapping, @PostMapping
 - RequestBody & ResponseEntity
 - CRUD API Development
 - Postman Testing
 - Swagger (API Documentation)
-

✔Step 11: Spring Data JPA

Advanced Hibernate with Spring

- Repositories (JpaRepository, CrudRepository)
 - Custom Query Methods
 - Pagination & Sorting
-

✔Step 12: Security with Spring Boot

Secure your applications

- Spring Security Basics
- Authentication vs Authorization
- Form-based login
- JWT (JSON Web Token) Authentication (Advanced)

✔Step 13: Android Development (Java)

Use Core Java to build mobile apps

- Android Studio Setup
- Activities and Intents
- UI Components (TextView, Button, EditText)
- RecyclerView, Adapters
- SQLite and Room Database
- SharedPreferences
- Calling REST APIs using Retrofit or Volley
- Firebase Integration (Login, DB, Notifications)

✔Step 14: Full Stack Java Development

Combine backend + frontend + database + deployment

Frontend Basics

- HTML, CSS, JavaScript
- React.js or Angular (optional but preferred)

Backend

- Spring Boot REST API

Database

- MySQL or PostgreSQL

Integration

- Connect Frontend to Backend (REST API)
- JWT Token Authentication

Deployment

- Deploy Java App on Heroku, Vercel, or Render
- Docker Basics (optional)
- CI/CD with GitHub Actions (optional)

✔ Step 15: Build Real-World Projects

Ideas:

- Student Management System (CRUD + DB)
- Online Quiz App (Spring Boot + REST API)
- Notes App (Android + Firebase)
- ECET Preparation App (Android + Web Backend)

🔗 Bonus Topics (Optional but Powerful)

- **Microservices with Spring Cloud**
- **Message Queues (RabbitMQ, Kafka)**
- **Unit Testing (JUnit, Mockito)**
- **Git/GitHub, CI/CD Pipelines**
- **Design Patterns in Java (Singleton, Factory, etc.)**

🔗 Optional but Powerful Topics in Java (Advanced Topics Roadmap)

☐ Stage 1: Testing and Version Control

1. JUnit (Java Unit Testing Framework)

- What is Unit Testing?
- JUnit Architecture
- Writing test cases using `@Test`
- Assertions (`assertEquals`, `assertTrue`, etc.)
- Test Suites
- Test Lifecycle (`@Before`, `@After`, etc.)

2. Mockito (Mocking Framework)

- What is Mocking?
- `@Mock`, `@InjectMocks`
- Stubbing and Verification
- Using Mockito with JUnit

3. Test-Driven Development (TDD)

- Red-Green-Refactor cycle

- Writing code through test cases
-

□ Stage 2: Build Tools and Project Automation

4. Maven

- pom.xml structure
- Adding dependencies
- Maven build lifecycle (clean, compile, package)
- Plugins and goals
- Maven project structure

5. Gradle (Optional alternative to Maven)

- build.gradle structure
 - Comparing Gradle vs Maven
 - Multi-module Gradle project
-

□ Stage 3: Version Control and CI/CD

6. Git & GitHub

- Git Basics: init, clone, add, commit, push, pull
- Branching and Merging
- Handling Merge Conflicts
- GitHub Issues, Pull Requests
- .gitignore, GitHub Repos

7. CI/CD (Continuous Integration/Deployment)

- What is CI/CD?
 - GitHub Actions Basics
 - Creating workflows (.yaml files)
 - Auto-build and test on every push
 - Jenkins (optional, enterprise use)
-

🧠 Stage 4: Microservices & Cloud Concepts

8. Microservices Architecture

- Monolithic vs Microservice
- Advantages of Microservices
- Communication (REST, FeignClient)
- Service Registry and Discovery
- Load Balancing

9. Spring Cloud Essentials

- Eureka Server (Service Registry)
- Spring Cloud Config (Central Config Server)
- API Gateway (using Spring Cloud Gateway)
- Feign Clients (Inter-service communication)
- Circuit Breaker (Resilience4j / Hystrix)

10. Docker (Basic Containerization)

- What is Docker and why use it?
- Dockerfile
- Building and Running Docker Images
- Dockerizing Spring Boot app

11. Docker Compose (Multi-container apps)

- `docker-compose.yml` file
- Running app + database together

□ Stage 5: Messaging and Real-Time Communication

12. RabbitMQ

- Message Queue Concept (Producer/Consumer)
- Installing RabbitMQ
- Queues, Exchanges, Bindings
- Integrating with Spring Boot

13. Apache Kafka

- What is Kafka?
- Topics, Producers, Consumers
- Kafka Brokers and Clusters
- Kafka + Spring Boot Integration

□ Stage 6: Design Patterns (Java Specific)

14. Creational Patterns

- Singleton Pattern
- Factory Pattern
- Abstract Factory
- Builder Pattern

15. Structural Patterns

- Adapter
- Proxy
- Composite
- Decorator

16. Behavioral Patterns

- Observer
- Strategy
- Command
- Template Method

17. SOLID Principles

- Single Responsibility
- Open/Closed
- Liskov Substitution
- Interface Segregation
- Dependency Inversion

□ Stage 7: Security Concepts

18. Spring Security (Advanced)

- Custom Login Pages
- In-Memory Authentication
- JWT (JSON Web Token) Authentication
- OAuth2 Basics
- Method-level Security (@PreAuthorize)

☛ Stage 8: Cloud Deployment & DevOps Basics

19. Deploy Spring Boot App

- Deploy on Render, Railway, or Heroku
- Using AWS EC2 (optional)
- Using Nginx as a reverse proxy

20. Monitoring & Logging

- Using Actuator
- Logback and SLF4J
- Prometheus + Grafana (Advanced)

📋 Final Roadmap Summary for Your Documentation

Stage	Topic Area
1	Unit Testing (JUnit, Mockito)
2	Build Tools (Maven, Gradle)
3	Git & GitHub, CI/CD (GitHub Actions, Jenkins)
4	Microservices (Spring Cloud, Eureka, Config Server)
5	Messaging Systems (RabbitMQ, Kafka)
6	Design Patterns and SOLID Principles
7	Security (Spring Security, JWT)
8	Deployment & Monitoring (Heroku, AWS, Logging)

Absolutely, Gopi! Here's the **complete roadmap of DSA topics** — from beginner to advanced — in **perfect order** for documentation and mastery, using **Java or any language**:

🔗 Complete DSA Topics (Beginner to Advanced)

☐ 1. Basics of Programming & Math for DSA

- Input/Output
 - Variables & Data Types
 - Conditionals and Loops
 - Functions & Recursion
 - Bit Manipulation
 - Modular Arithmetic
 - GCD, LCM, Prime Numbers, Sieve of Eratosthenes
 - Euclidean Algorithm
-

☐ 2. Time & Space Complexity

- Big O, Big Ω , Big Θ
 - Best, Worst, Average Cases
 - Time-Space Tradeoffs
-

☐ 3. Arrays

- 1D & 2D Arrays
 - Prefix Sum, Sliding Window
 - Kadane's Algorithm
 - Subarrays & Ranges
 - Sorting (Bubble, Selection, Insertion, Merge, Quick, Count, Radix)
-

☐ 4. Strings

- String Basics, Immutability
 - StringBuilder/StringBuffer
 - Pattern Matching (Naive, KMP, Rabin-Karp)
 - Palindromes
 - Anagram, Substring, Rotation
 - Z-Algorithm, Suffix Array (Advanced)
-

☐ 5. Searching & Binary Search

- Linear Search
 - Binary Search
 - Binary Search on Answer
 - Lower Bound / Upper Bound
 - Order-Agnostic Search
-

□ 6. Recursion & Backtracking

- Basic Recursion
 - Factorial, Fibonacci, Tower of Hanoi
 - N-Queens Problem
 - Sudoku Solver
 - Subsets, Permutations, Combinations
-

□ 7. Hashing

- HashMap, HashSet
 - Frequency Counter
 - Two Sum / Subarray Sum
 - Hashing with Chaining & Open Addressing
 - Count Distinct Elements
 - Group Anagrams
-

□ 8. Stack

- Stack Implementation
 - Next Greater Element
 - Balanced Parentheses
 - Infix to Postfix / Prefix
 - Histogram Maximum Area
-

□ 9. Queue & Deque

- Queue Implementation (Array, LinkedList)
- Circular Queue
- Sliding Window Maximum
- LRU Cache (with Deque + HashMap)

- Monotonic Queue
-

□ 10. Linked List

- Singly, Doubly, Circular Linked List
 - Reverse Linked List
 - Detect and Remove Loop (Floyd's Cycle)
 - Merge Two Sorted Lists
 - Intersection Point
 - LRU Cache (LinkedList + HashMap)
-

□ 11. Trees

- Binary Tree, BST
 - Traversals (Inorder, Preorder, Postorder, Level Order)
 - Height, Diameter
 - Lowest Common Ancestor
 - Balanced Tree Check
 - AVL Tree, Segment Tree (Advanced)
 - Binary Indexed Tree (Fenwick Tree)
-

□ 12. Heap / Priority Queue

- Min Heap / Max Heap
 - Heapify, Extract Min/Max
 - K Largest/Smallest Elements
 - Median of Stream
 - HeapSort
-

□ 13. Trie (Prefix Tree)

- Insert/Search Word
- Word Suggestions (Autocomplete)
- Word Break Problem
- Count Prefixes
- Advanced: Trie + Bit Manipulation

□ 14. Graphs

- Representation (Adjacency List/Matrix)
 - BFS, DFS
 - Cycle Detection (Directed/Undirected)
 - Connected Components
 - Topological Sort
 - Dijkstra's Algorithm
 - Bellman-Ford
 - Floyd Warshall
 - MST (Prim's, Kruskal's)
 - Disjoint Set (Union-Find)
-

□ 15. Dynamic Programming (DP)

- Recursion → Memoization → Tabulation
 - 0/1 Knapsack, Unbounded Knapsack
 - Longest Common Subsequence (LCS)
 - Longest Increasing Subsequence (LIS)
 - Matrix Chain Multiplication
 - DP on Trees, DP on Graphs
 - Bitmask DP
 - Digit DP
-

□ 16. Greedy Algorithms

- Activity Selection
 - Fractional Knapsack
 - Job Scheduling
 - Huffman Encoding
 - Minimum Platforms
-

□ 17. Sliding Window & Two Pointers

- Fixed & Variable Size Windows
- Max Sum Subarray
- Longest Substring Without Repeat

- Count of Subarrays with Conditions
-

□ 18. Bit Manipulation

- Set/Unset/Toggle Bits
 - Count Set Bits
 - Power of Two
 - XOR Operations
 - Bitwise Tricks in Problems
-

□ 19. Advanced Topics (For Competitions / Interviews)

- Mo's Algorithm
- Sqrt Decomposition
- Centroid Decomposition
- Segment Tree Beats
- Persistent Segment Tree
- KMP + Z-Algorithm + Manacher's
- Heavy-Light Decomposition (HLD)
- Graph Coloring, Articulation Points

🔍 Optional but Powerful Advanced DSA Topics:

🔍 1. *Advanced Number Theory*

- Modular Arithmetic (Addition, Subtraction, Multiplication)
 - **Modular Inverse**
 - **Extended Euclidean Algorithm**
 - **Fermat's Little Theorem**
 - **Chinese Remainder Theorem**
 - **Euler's Totient Function**
-

🔍 2. *Game Theory*

- Grundy Numbers / Sprague-Grundy Theorem
- Nim Game, Subtraction Game
- Impartial vs Partisan Games
- Minimax Algorithm
- Alpha-Beta Pruning (also used in AI)

3. Matrix Algorithms

- Matrix Multiplication (Naive and Strassen)
- **Matrix Exponentiation** (used in solving linear recurrences)
- Applications in Fibonacci, Combinatorics, and Graphs

4. Dynamic Connectivity

- Union-Find with Rollbacks
- Dynamic MST
- Link-Cut Trees (*Very advanced*)
- Euler Tour Trees

5. Advanced Tree Algorithms

- **Heavy-Light Decomposition (HLD)**
- **Centroid Decomposition**
- **Binary Lifting for LCA (Lowest Common Ancestor)**
- **Euler Tour + Segment Tree**
- Tree DP (on rooted and unrooted trees)

6. Suffix-Based Structures

- **Suffix Tree**
- **Suffix Array**
- **Longest Common Prefix (LCP) Array**
- **Generalized Suffix Automaton** (*Very advanced, used in string matching*)

7. Bitmask DP

- Subset DP
 - Traveling Salesman Problem (TSP) using Bitmask
 - DP over permutations
 - State compression techniques
-

8. Combinatorics & Math DS

- Factorials & nCr Modulo Prime
 - Lucas Theorem
 - Inclusion-Exclusion Principle
 - Fast Power / Binary Exponentiation
 - Burnside Lemma (Group Theory, rare)
-

9. Miscellaneous Pro-Level Algorithms

- Mo's Algorithm (Query Square Root Decomposition)
 - Convex Hull Trick (for DP optimization)
 - Persistent Data Structures (Segment Trees, Trie)
 - Li Chao Tree
 - Treaps / AVL / Splay Trees (Self-balancing BSTs)
-

□ **Note:** These topics are rarely asked in interviews but are gold for **CP, ICPC, or top-level contests**.